

Claude Rulebook

Applied principles from Thariq's (@trq212) Claude Code essays
and the Claude Agent SDK guide

Drop this PDF into any Claude project (via /project or CLAUDE.md reference) so the agent applies the same context, caching, tool-design and session-management discipline that the Claude Code team uses internally.

How to use this document

This is a rulebook, not prose. Each rule is self-contained and actionable. When Claude reads this file, it should treat every numbered rule as a directive applicable to the current project. Rules are grouped by topic; the final section is a one-page quick reference.

Sources (all by Thariq / @trq212 unless noted)

- Lessons from Building Claude Code: Seeing like an Agent
- Lessons from Building Claude Code: Prompt Caching Is Everything
- Making Playgrounds using Claude Code
- Your Agent should use a File System
- Why even non-coding agents need bash
- Spec-based development with Claude Code
- Using Claude Code: Session Management & 1M Context
- Building Agents with the Claude Agent SDK (Claude blog)

1. The Agent Loop

Every serious agentic task should cycle through four phases. Skipping the verify phase is the single biggest cause of compounding errors.

Phase	What happens	Failure mode if skipped
Gather context	Read files, run grep, query docs, load only what is needed	Hidden assumptions, wrong file edits
Take action	Edit code, run scripts, call tools	N/A (always present)
Verify work	Run tests, re-read diff, screenshot, lint, LLM-as-judge	Compounding errors, silent regressions
Repeat	Feed verification back into the next gather phase	Agent drifts further from the goal each turn

Rules

- R1.1 — Before touching code, list the files/facts you intend to read; if the list is empty, ask before acting.
- R1.2 — After every non-trivial action, verify it explicitly: run a test, read the diff back, take a screenshot, or run a lint pass. Never end a turn on an unverified mutation.
- R1.3 — If a verification fails twice in the same approach, stop and rewind instead of patching. A third attempt on the same path almost always compounds the bug.
- R1.4 — Prefer code-level feedback (linter, typecheck, unit test) over free-text self-review; it is faster and cheaper to trust.
- R1.5 — For UI/layout work, verification means a render or a screenshot — not a description.

2. Context & Session Management

Context is a resource, not a free parameter. Performance degrades (context rot) roughly between 300–400k tokens on a 1M model, task-dependent. A new task means a new session by default.

The five-option decision tree after every turn

Option	When to use it
Continue	Same task, context still clean and relevant.
/rewind (esc esc)	An approach failed. Jump back to after the file reads and re-prompt with what you learned — do NOT
/clear	New task OR context is polluted. Write a short brief from what you learned and start fresh.
/compact	Long session, same task. Always pass a directive (e.g. "focus on the auth refactor, drop the test del
Subagent	The next chunk will produce intermediate output you will not need again. You only want the conclus

Rules

- R2.1 — Rewind beats correction. If the approach fails, do not say "that didn't work, try X". Rewind to just after the file reads and re-prompt with the learning explicit.
- R2.2 — New task = new session. Exception only when re-reading recent files would cost more than the context pollution (e.g., documenting a feature you just implemented).
- R2.3 — Compact proactively, not reactively. Trigger /compact with a directive BEFORE context rot sets in — never let autocompact fire at the end of a long debugging run.
- R2.4 — A bad compact happens when the model can't predict the next direction. Pre-declare the next task inside the compact directive.
- R2.5 — Do not run subagents for 2–3 lookups. Grep and Read are faster and cheaper. Subagents pay off when the work would otherwise dump intermediate tool output you'll never reuse.
- R2.6 — Use "summarize from here" (or a handoff file) to let Claude write a message from its future self to its past self — especially before a rewind.
- R2.7 — Keep an open session alive only when context is still load-bearing. "It's faster to continue" is not a reason; measure it against rewind + fresh.

3. Prompt Caching

Prompt caching is a prefix match. Any change anywhere in the cached prefix invalidates everything after it. If you are building a long-running or multi-turn agent, the entire harness must be designed around this constraint — it is a correctness concern, not just an optimization.

Recommended prefix layout (static → dynamic)

Layer	Content	Cache scope
1	System prompt + tool definitions	Global (shared across all sessions)
2	CLAUDE.md / project-level instructions	Per project
3	Session context / loaded files	Per session
4	Conversation messages / tool results	Per turn

Rules

- R3.1 — Order the prefix static → dynamic. Any timestamp, random tool order, or changing param in the system prompt invalidates everything downstream.
- R3.2 — Never put a precise timestamp in the system prompt. If the model needs the current time, inject it as a <system-reminder> in the next user message or tool result.
- R3.3 — Never change models mid-session for minor cost savings. The new model rebuilds cache from scratch; that is usually more expensive than letting the current model finish.
- R3.4 — If you must change models, do it via subagent handoff. Let the big model write a task description, then spawn a smaller model with its own fresh context.
- R3.5 — Never add or remove tools mid-session. Model state transitions (plan mode, read-only mode, review mode) as TOOLS (e.g. EnterPlanMode / ExitPlanMode) — not by swapping the tool list.
- R3.6 — When you have many optional tools, use defer_loading stubs: include tool names in the cached prefix, load full schemas only when the model selects them.
- R3.7 — Compaction must reuse the parent's prefix. Same system prompt, same tools, same history — then append the compact prompt as a new user message at the end.
- R3.8 — Monitor cache hit rate like you monitor uptime. A few percentage points of miss rate can blow up cost and latency.

4. Tool Design (“Seeing like an Agent”)

Tools are the agent's action space. Design them for the model's abilities, not for your mental model of "what a user would want". The bar to add a new tool is high because every tool costs attention in every turn.

Rules

- R4.1 — Match tools to the model's ability, like matching paper vs. calculator vs. computer to a student. If the agent can code, bash + a file system beat twenty narrow tools.
- R4.2 — High bar for new tools. Every tool adds one more option the model must consider on every turn. Prefer: a new skill, progressive disclosure, or a subagent.
- R4.3 — Progressive disclosure > stuffing the system prompt. Let the agent navigate from a short root instruction into deeper files on demand (skills pattern).
- R4.4 — Elicitation is a first-class capability. Use a dedicated AskUserQuestion tool with structured options — never rely on free-text "please clarify".
- R4.5 — Re-examine tools as model capability grows. A tool that helped yesterday (e.g., a rigid todo list) may constrain a smarter model tomorrow. Revisit the set quarterly.
- R4.6 — When the agent misunderstands a task, do not add a new tool first. Restructure the existing tool's input/output so the information is easier to discover.
- R4.7 — Agentic search beats RAG for transparency and maintainability. Start with grep/read. Only add vector search when you have hard latency/scale evidence.
- R4.8 — If the agent repeatedly fails at a task, add formal rules to the tool description ("you must X before Y") before you build new machinery.

4b. Bash as the universal tool

Even non-coding agents benefit from bash. It handles file I/O, format conversion, selective loading of big files (grep / head / tail), downloads, and ad-hoc data slicing that would otherwise require five specialized tools.

- R4.9 — Every agent gets a bash (or equivalent) tool. Email agents, research agents, comptax agents — all of them.
- R4.10 — Use bash to pre-filter large content into context (grep, tail, head, jq). Never dump an unbounded log or CSV into the model's window.
- R4.11 — When stuck on "I don't have a tool for X", ask first: can bash + a standard CLI do it? (curl, ffmpeg, jq, pandoc, awk, rg).

5. File System & State

Every agent should use a file system. It is the simplest durable representation of state, it is trivial to inspect, and it is how the agent can verify its own work between turns.

- R5.1 — Persist decisions, specs, plans, and handoffs in files — never keep them only in the conversation buffer. Sessions die; files survive.
- R5.2 — Folder and file naming are context engineering. A tree like specs/, plans/, reports/, out/ tells the agent where to look before you do.
- R5.3 — For large artifacts (logs, data dumps, uploaded PDFs), keep them on disk and load slices via bash — not via big single-shot reads.
- R5.4 — Every non-trivial project keeps a CACHE.md (or equivalent) with the current state, last actions, and open questions. Read it first, update it last.
- R5.5 — Handover between agents/sessions is a compaction: write a file that includes decisions made, blockers, dependencies, next action. Next agent reads that file first.
- R5.6 — Never write sensitive data (credentials, tokens, PII) into a working file that may be read into context by another session or agent.

5b. Verification feedback loops

- R5.7 — Rules-based feedback first (lint, typecheck, test). Deterministic, fast, cheap.
- R5.8 — Visual feedback when the output is visual (screenshots, rendered HTML, Playwright). Text-only review of layout work is theater.
- R5.9 — LLM-as-judge only when rules-based and visual feedback don't apply. Expect latency and be explicit about the rubric.
- R5.10 — Prefer TypeScript over JavaScript, typed schemas over dicts — it doubles the layers of automatic feedback.

6. Workflow Patterns

6a. Spec-based development

Build large features in two phases in two different sessions. Do not try to clarify and execute in the same context — it pollutes the execution session with exploration noise.

- R6.1 — Phase 1 (planning session): give Claude a minimal prompt. Ask it to interview you using AskUserQuestion until the spec is unambiguous. Output: a spec file.
- R6.2 — Phase 2 (execution session): /clear (or a fresh session). First action = read the spec file. Execute.
- R6.3 — If during execution you discover a gap in the spec, do NOT patch it in the execution session. Write down the gap, go back to the planning session, update the spec, restart execution.
- R6.4 — The spec is the contract. If it is wrong, the execution output is wrong even if the code runs.

6b. Subagents

- R6.5 — Subagent test: "Will I need this tool output again, or just the conclusion?" If conclusion only → subagent.
- R6.6 — Use subagents for: verification against a spec; reading another codebase to summarize a pattern; drafting docs from git diffs; multi-source research; independent cross-checks.
- R6.7 — Do NOT use subagents for: a handful of greps; a single file read; a sequence where each step depends on the previous one's raw output.
- R6.8 — Every subagent prompt is self-contained. Include the goal, what has been tried, what files/specs matter, and the expected shape of the answer. The subagent does not see the parent conversation.
- R6.9 — Spawn parallel subagents in a single message when they are independent (research left, research right, verify spec). Never sequence independent subagents.

6c. Playgrounds (visual iteration)

When text is a bad fit for the problem — design, layout, game balance, architecture diagrams, data exploration — ask Claude to generate a standalone HTML playground you can interact with. Copy the output back as the next prompt.

- R6.10 — Reach for a playground whenever the feedback needs a picture, not a paragraph. Layouts, graphs, balance tuning, architecture diagrams, inline doc review.
- R6.11 — Playground outputs should emit a "paste-back prompt" the user can drop into the next turn. Close the loop automatically.
- R6.12 — Playgrounds are disposable. Commit the skill/prompt, not the generated HTML.

7. Building with the Claude Agent SDK

The Agent SDK gives you the same primitives the Claude Code team uses internally. Do not reinvent them — use them, and let the SDK handle the boring correctness work (auth, MCP, compaction, retries).

- R7.1 — Default harness = bash + file system + small focused tools. Start there, only add more if a real failure mode demands it.
- R7.2 — Use MCP for anything external. Never hand-roll OAuth or REST glue; prefer a prebuilt MCP server or write a FastMCP one.
- R7.3 — Turn repeating guidelines into a skill (SKILL.md) so they load lazily. Do not bloat the system prompt with every domain rule.
- R7.4 — Treat "code" as an output format. Anytime the answer can be expressed as reusable code, prefer that over prose — it is precise, composable, and testable.
- R7.5 — Build a small evals set as soon as the agent does something real. Any new feature is shipped against that eval, not vibes.
- R7.6 — Automatic compaction on: long-running agents must never die from running out of tokens. The SDK can handle this; opt in.
- R7.7 — When the agent is failing, read its outputs like it is a junior colleague. What did IT see? Not what did you hope it would see.

8. Quick Reference Card

Print this page. Keep it next to the keyboard.

Situation	Do this
Starting any non-trivial task	Ask clarifying questions (AskUserQuestion) before acting
Approach failed twice	/rewind to just after file reads, re-prompt with the learning
Starting a new task	/clear + brief distilled from what you just learned
Session getting long, same task	/compact with a directive ("focus on X, drop Y")
Next chunk = lots of intermediate output	Spawn a subagent, keep only the conclusion
Need current date/time in prompt	Inject via <system-reminder> in next message, NOT system prompt
Want to enter plan/review mode	Use a tool (EnterPlanMode) — do not change the tool list
Many optional tools	defer_loading stubs in prefix, full schemas on demand
Verifying layout/UI work	Render or screenshot — never a text description
Running into "I don't have a tool for X"	Try bash + a standard CLI first
Building a large feature	Spec session → fresh execution session
Big ambiguous design/balance problem	Generate an HTML playground, iterate visually
Persisting anything across turns	Write to a file. CACHE.md / specs/ / plans/
Shipping a new feature	Extend the evals set before merging
Cache hit rate dropped	Treat as an incident. Diff the prefix.

When in doubt: **read the file, verify the result, rewind on failure, keep the cache stable.**

Addendum — Essays Published After v1

This addendum extends the rulebook with three essays by Thariq (@trq212) published after the original synthesis: **Lessons from Building Claude Code: How We Use Skills** (Mar 17, 2026), **Using Claude Code: The Unreasonable Effectiveness of HTML** (May 8, 2026), and **A Harness for Every Task: Dynamic Workflows in Claude Code** (Jun 2, 2026). Rules follow the same numbering convention and are directives for the current project.

9. Skills (“How We Use Skills”)

Skills are one of the most-used extension points in Claude Code. They are not “just markdown files” — they are folders that can include scripts, assets, data, config, and hooks the agent can discover, explore, and manipulate. The best skills fit cleanly into one category; confusing ones straddle several.

The 9 skill categories

Category	Example
1. Library & API reference	billing-lib edge cases, internal CLI subcommands, design system
2. Product verification	signup-flow-driver, checkout-verifier, tmux-cli-driver
3. Data fetching & analysis	funnel-query, cohort-compare, grafana dashboard lookups
4. Business process automation	standup-post, create-ticket with schema, weekly-recap
5. Code scaffolding & templates	new-migration, create-app with auth/logging pre-wired
6. Code quality & review	adversarial-review, code-style, testing-practices
7. CI/CD & deployment	babysit-pr, deploy-service with auto-rollback, cherry-pick-prod
8. Runbooks	service-debugging, oncall-runner, log-correlator
9. Infrastructure operations	resource-orphans cleanup with soak period, cost-investigation

Rules

- **R9.1** — A skill is a folder, not a markdown file. Use scripts, reference snippets, assets, templates and data files inside it; tell Claude what files exist so it reads them at the right time.
- **R9.2** — Don't state the obvious. A knowledge skill must push Claude OUT of its default opinions (e.g. frontend-design banning Inter font + purple gradients) — not restate what it already does.
- **R9.3** — The Gotchas section is the highest-signal content of any skill. Build it from real failure points and keep appending as Claude hits new edge cases.
- **R9.4** — Use progressive disclosure: split detailed function signatures into references/api.md, put output templates in assets/, keep SKILL.md short.
- **R9.5** — Avoid railroading. Give Claude the information it needs plus the flexibility to adapt; over-specific instructions break on reuse.
- **R9.6** — Skills needing user context store it in a config.json inside the skill directory; if missing, the agent asks (via AskUserQuestion for structured choices).

- **R9.7** — The description field is for the model: it is a TRIGGER CONDITION (“when to use this”), not a summary. It is what Claude scans to decide “is there a skill for this request?”
- **R9.8** — Skills can have memory: an append-only log, JSON, or SQLite. Store persistent data in a stable folder (e.g. `$(CLAUDE_PLUGIN_DATA)`) — skill-directory data may be wiped on upgrade.
- **R9.9** — Ship scripts and helper libraries inside the skill so Claude spends its turns composing, not reconstructing boilerplate.
- **R9.10** — Use on-demand hooks for opinionated guards you only want sometimes (/careful blocking `rm -rf` and `DROP TABLE`; /freeze blocking edits outside one directory).
- **R9.11** — Verification skills deserve dedicated engineering time. Record videos of output, enforce programmatic assertions at each step.
- **R9.12** — Distribution: check skills into the repo (`./claude/skills`) for small teams; use a plugin marketplace at scale. Curate — promote skills organically after they get traction, never auto-accept.
- **R9.13** — Compose skills by referencing other skills by name in the `SKILL.md`; the model will invoke them if installed.
- **R9.14** — Measure skill usage (e.g. a `PreToolUse` logging hook) to find popular and under-triggering skills.

10. HTML as Output Format (“The Unreasonable Effectiveness of HTML”)

Markdown becomes a restricting format once agents write large specs, plans and reports: nobody reads a 100+ line markdown file. HTML offers information density (tables, CSS, SVG, interactions, spatial data), visual clarity, easy sharing (a link beats an attachment), and two-way interaction. The cost: 2–4× longer generation and noisy diffs in version control.

Rules

- **R10.1** — Prefer HTML over markdown for any substantial artifact a human must read: specs, plans, reports, PR explainers, research syntheses. Reserve markdown for short, diff-reviewed content.
- **R10.2** — No /html skill needed. Just prompt “make an HTML file” and specify what the artifact should do and how it will be used.
- **R10.3** — Planning is a WEB of HTML files: brainstorm explorations → expand one with mockups and code snippets → implementation plan → new session reads all files and implements. Verification agents read the same files for broader context.
- **R10.4** — Code review: render the actual diff with inline margin annotations, color-coded findings by severity, flowcharts of modules. Attach an HTML explainer to every non-trivial PR.
- **R10.5** — Design & prototyping: sketch in HTML even when the target is React/Swift. Ask for sliders/knobs to tune animations and parameters interactively.
- **R10.6** — Reports: synthesize Slack + codebase + git history + web into a single readable HTML page with SVG diagrams; optimize for someone reading it once.
- **R10.7** — Custom throwaway editors: when the input is painful to express in text (reordering tickets, feature flags, prompt tuning, colors, cron schedules), ask for a single-purpose HTML editor.
- **R10.8** — ALWAYS end interactive artifacts with an export: a “copy as JSON / copy as prompt” button that turns UI state back into something pasteable into the next turn.
- **R10.9** — Keep taste consistent: generate one design-system HTML file from your codebase and use it as the reference for all other HTML artifacts.
- **R10.10** — Know the trade-offs: HTML costs more tokens and renders noisy diffs. For version-controlled prose that must be reviewed line-by-line, markdown still wins.

11. Dynamic Workflows (“A Harness for Every Task”)

Claude can write its own harness on the fly: a JavaScript file with special functions that spawn and coordinate subagents, choose each agent's model and worktree isolation, and resume if interrupted. Workflows combat three single-context failure modes: **agentic laziness** (declaring done after partial progress), **self-preferential bias** (preferring its own findings when judging), and **goal drift** (lossy compaction dropping constraints).

The 6 composable patterns

Pattern	Use when
Classify-and-act	Route to different agents/behavior based on task type
Fan-out-and-synthesize	Many small steps, each wants a clean context; synthesis is the barrier
Adversarial verification	Each worker agent gets a separate verifier against a rubric
Generate-and-filter	Generate many ideas, filter by rubric/verification, dedupe, keep the best
Tournament	Agents compete on the same task; pairwise judging picks a winner
Loop until done	Unknown amount of work; loop agents until a stop condition is met

Rules

- **R11.1** — Use a workflow when the task is long-running, massively parallel, or adversarial/structured — NOT for regular coding tasks. Ask: “does this really need more compute?” Most tasks don't need a panel of 5 reviewers.
- **R11.2** — Trigger explicitly (“use a workflow” / “ultracode”) and prompt in detail using the pattern names above. “Quick workflows” are valid for small adversarial checks.
- **R11.3** — Migrations/refactors: break into units (callsites, failing tests, modules), one subagent per fix in its own worktree, adversarial review agent, then merge. Bun's Zig→Rust rewrite used this.
- **R11.4** — Deep verification: one agent extracts every factual claim, one subagent checks each claim, a third checks source quality.
- **R11.5** — Sorting 1000+ items by qualitative judgment: never one prompt. Run a tournament, pairwise-comparison pipeline (comparative > absolute scoring), or parallel bucket-rank then merge.
- **R11.6** — Rule adherence: one verifier agent per rule, plus a skeptic persona reviewing findings to kill false positives. Reverse direction: mine sessions for repeated corrections, cluster, adversarially verify, distill survivors into CLAUDE.md.
- **R11.7** — Root-cause investigation: spin up hypothesis agents on DISJOINT evidence (logs / files / data), then a panel of verifiers and refuters. Works for sales drops and pipeline failures, not just code.
- **R11.8** — Triage at scale: classify → dedupe → act/escalate. Apply QUARANTINE: agents reading untrusted public content must not hold high-privilege actions; acting agents do.
- **R11.9** — Taste/exploration: generate many solutions, give a review agent an explicit rubric, stop when criteria are met or run a tournament.
- **R11.10** — Model routing: a classifier agent researches the task shape first, then routes to Sonnet or Opus based on expected complexity.
- **R11.11** — Set token budgets explicitly (“use 10k tokens”). Workflows use more tokens; budget them like money.

- **R11.12** — Pair recurring workflows (triage, research, verification) with /loop for regular intervals and /goal for hard completion requirements.
- **R11.13** — Save good workflows (press “s”, or ~/.claude/workflows) and distribute via a skill — but frame them as TEMPLATES the model can adapt, not scripts to run verbatim.
- **R11.14** — Evals-by-workflow: spin agents in worktrees, then comparison agents grading outputs against a rubric (e.g. refining a skill against criteria).

Quick Reference — Additions

Situation	Do this
Repeating guidance Claude keeps missing	Make/extend a skill; put failures in the Gotchas section
Skill exists but never triggers	Rewrite the description field as a trigger condition
Human must actually read the output	HTML file, not markdown; SVG diagrams; share a link
Interactive artifact built	Add a “copy as prompt/JSON” export button
Huge parallel/adversarial/structured task	Dynamic workflow (“ultracode”) + token budget
1000+ items to rank	Tournament / pairwise comparison agents, never one prompt
Verifying agent judges its own work	Separate adversarial verifier agent (fresh context)
Workflow reads untrusted content	Quarantine: readers get no high-privilege actions
Workflow worth keeping	Save to ~/.claude/workflows or ship in a skill as a template